

使用Python连接MySQL数据库

第17章 使用Python连接MySQL数据库

- ❑ MySQL已经成为最流行的关系型数据库管理系统之一，具有稳定、可靠、速度快、易于设置、使用和管理的特点，可以在各种版本的UNIX和Windows环境下使用，为多种编程语言提供了API，如C、C++、Python、Java、Perl、PHP等。
- ❑ Python是一种解释型、面向对象、动态数据类型的高级程序设计语言，支持命令式编程、函数式编程，拥有大量功能强大的内置对象、标准库。Python在大数据、机器学习、人工智能等领域发展迅速，同时也适用于Web开发、后端、移动应用程序开发等。Python提供了多种操作MySQL数据库的方式，可以适合不同类型项目需求的需要。本章将介绍Python对MySQL数据库操作的原理。



STEP 01

Python
程序语言



STEP 02

Python访问
MySQL数据库概述



STEP 03

Python连接
MySQL编程实践

Python 程序语言

CONTENTS

This Is A
List

01

Python概述

02

Python的应用场景

01

PART ONE

Python概述

□ Python诞生于20世纪90年代，目前已经逐渐广泛应用于系统管理任务的处理和Web编程。由于Python的简洁性、易读性和可扩展性，还提供了大量的第三方扩展库，如Pandas、Numpy、Matplotlib、Scripy、Keras等，Python逐渐在数据分析、机器学习、人工智能等领域占据越来越重要的位置。

□ 与C、C++、Java等程序语言相比，Python易学易用，语法简洁清晰，代码可读性强。自2004年以来，Python的使用率呈线性增长。2020年9月，Python被TIOBE编程语言排行榜评为第三名，目前已经第一名。

Mar 2022	Mar 2021	Change	Programming Language
1	3	▲	 Python
2	1	▼	 C
3	2	▼	 Java
4	4		 C++
5	5		 C#
6	6		 Visual Basic
7	7		 JavaScript
8	8		 PHP

Python 发行版本与时间

版本	发布时间	最后更新时间	最后版本
Python 1	1994年发布	已停止	
Python 2.x	2000年10月	2020 年 1 月 1 日	Python 2.7
Python 3.x	2008年12月3日	依然在更新	Python3.9.2

本章将以Python 3.x为例进行叙述。



02

PART Three

Python 优势和应用场景

□ python是一种面向对象的计算机程序设计语言。Python具有强大的库，它常被称作胶水语言，能够把使用其他语言编写的各种模块（尤其是C/C++）很轻松的联结在一起，主要包含以下优势：

- 编程简单。相比其他编程语言（如 Java），Python 代码简单，上手容易。当我们要完成某个功能，如果用 Java 需要 100 行代码，但用 Python 可能只需要 20 行代码。
- Python语法清晰。它不是一种格式自由的语言，语法格式清晰。例如，它要求 if 语句的下一行必须向右缩进，否则不能通过编译。
- 可扩展性好。Python 的可扩展性体现为它的模块，Python 具有脚本语言中最丰富和强大的类库，这些类库覆盖了文件 I/O、GUI、网络编程、数据库访问、文本操作等绝大部分应用场景。
- Python具有很强的面向对象特性。它消除了保护类型、抽象类、接口等面向对象的元素。

❑ Python是目前最为流行的编程语言之一，在Web开发、大数据开发、人工智能开发、嵌入式开发等领域均有广泛的应用，同时也是网络运维人员比较常见的编程工具之一。

- **Web开发。**与PHP和Java一样，Python也经常应用于Web开发，并且提供了较为方便的框架，如Flask, Django, TurboGrars等。
- **人工智能领域。**Python在人工智能领域内的机器学习、神经网络、深度学习等方面应用广泛。目前，基于大数据分析和深度学习发展的人工智能学习框架，例如，神经网络框架TensorFlow、PyTorch以及开源的 Karas 神经网络库等，都是用 Python 实现的。

- 嵌入式领域。Python语言打通了整个物联网开发体系，从设备、网络、平台到分析和应用，整个物联网开发体系都可以采用Python语言来完成功能开发。
- 自动化运维。在很多操作系统中，Python是标准的系统组件，如NetBSD、OpenBSD 和 Mac OS X 等都集成了 Python，可以在终端下直接运行 Python。另外，Python标准库中包含了多个可用于调用操作系统功能的库，通常情况下，Python语言编写的系统管理脚本，在可读性、性能、代码重用度以及扩展性方面，都优于普通的Shell脚本。
- 服务器软件。Python支持各种网络协议，经常被用于编写服务器软件、网络爬虫，游戏软件等。

Python访问

MySQL数据库概述

CONTENTS

This Is A
List

01

Python数据库访问工具概述

02

使用Python DB API访问数据库的流程

03

Python连接MySQL数据库

04

Python连接数据库环境搭建

01

PART ONE

数据库访问工具

- ❑ Python中所有数据库接口程序都在一定程度上遵守Python DB API规范。Python DB API是Python访问数据库的统一接口规范。
- ❑ 它定义了一系列的对象和数据库存取方式，可以为各种各样的底层数据库系统和多种多样的数据库接口程序提供一致的访问接口。
- ❑ 使用Python DB API连接数据库后，就可以用相同的方式操作各种数据库，从而轻松实现不同的数据库之间的代码移植。

❑ Python DB API主要包括数据库连接对象Connection，数据库交互对象Cursor和数据库异常类Exceptions。

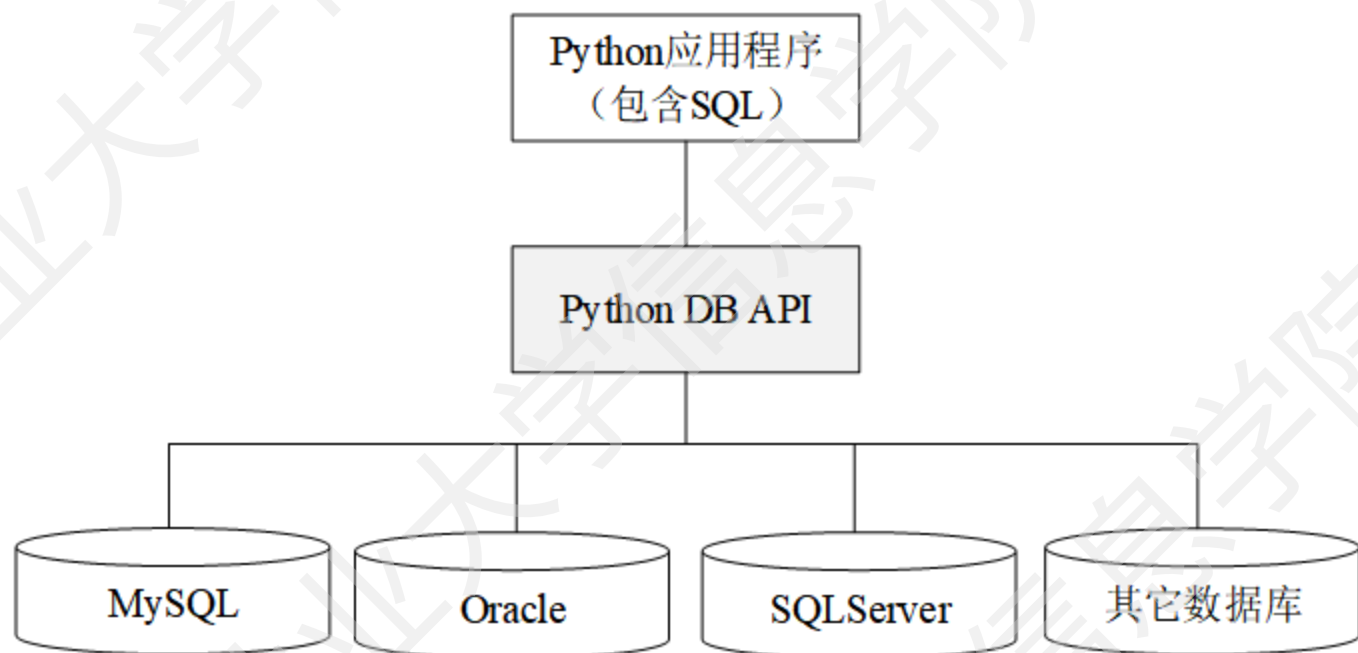


图17-1 Python 访问数据库的统一接口

- ❑ Connection用于与MySQL数据库系统建立连接，等价于到服务器的实际网络连接，其主要方法如表17-1所示。

方法	说明
<code>cursor()</code>	使用连接返回一个新的游标对象
<code>commit()</code>	提交当前事务，如果不提交，那么自上次调用 <code>commit()</code> 方法之后的所有修改都不会保存到数据库文件中
<code>rollback()</code>	撤销当前事务，此方法将使得数据库回滚到提交 <code>commit()</code> 方法后的状态。在未提交的情况下关闭连接将导致执行隐式回滚
<code>close()</code>	关闭数据库连接，即从现在开始，Python与MySQL服务器的连接无法使用。如果试图对该连接进行任何操作，将引发Error异常

表17-1 Connection对象的主要方法

- ❑ 虽然通过连接对象Connection可以访问数据库，但是接口模块必须为上述方法提供构造函数`connect(parameters..)`来创建到数据库的连接，返回值为一个连接对象。该函数接受许多依赖于数据库的参数。

- ❑ Python可以利用Cursor对象向MySQL发送SQL查询，以及获取MySQL处理查询生成的结果，Cursor支持的主要方法如表17-2所示。

方法	说明
execute(query [,args])	执行一个数据库查询和命令，其中query（字符型）参数为需要执行的查询操作，args（元组、列表或字典类型），可选，为与查询一起使用的参数。
nextset()	将游标跳到下一个可用集，丢弃当前集中的任何剩余行。如果没有更多的集合，该方法返回None
fetchone()	获取结果集的下一行
fetchmany(size)	获取结果集的指定几行
fetchall()	获取结果集中的所有行
rowcount	最近一次execute返回数据的行数或影响行数
close()	关闭游标对象

表17-2 Cursor对象的主要方法

- ❑ Python的DB API 通过Exceptions异常类或其子类提供所有的错误信息，在Python操作数据库过程中主要包括以下异常错误。

异常	描述
Warning	严重警告引发的异常，例如插入数据是被截断等，它必须是 StandardError 的子类
Error	所有其它错误异常的基类，可以捕获警告以外所有其他错误类，必须是 StandardError 的子类
InterfaceError	数据库接口模块的错误（而不是数据库的错误）引发的异常，必须是Error的子类
DatabaseError	与数据库有关的错误引发的异常，必须是Error的子类
DataError	处理数据时的错误引发的异常，例如：除零错误，数据超范围等， 必须是 DatabaseError的子类

表17-3 Exceptions异常错误

OperationalError	与数据库操作相关且不一定在用户控制下发生的错误引发的异常，例如，连接意外断开、找不到数据源名称、事务无法处理，必须是DatabaseError的子类
IntegrityError	数据库的关系完整性相关的错误引发的异常，例如外键检查失败等，必须是DatabaseError子类
InternalError	数据库的内部错误引发的异常，例如游标（cursor）不再有效、事务不同步等，必须是DatabaseError子类
ProgrammingError	程序错误引发的异常，例如SQL语句语法错误、参数数量错误等，必须是DatabaseError的子类
NotSupportedError	使用数据库不支持的方法或API引发的异常，例如，使用.rollback()函数作为连接函数，必须是DatabaseError的子类



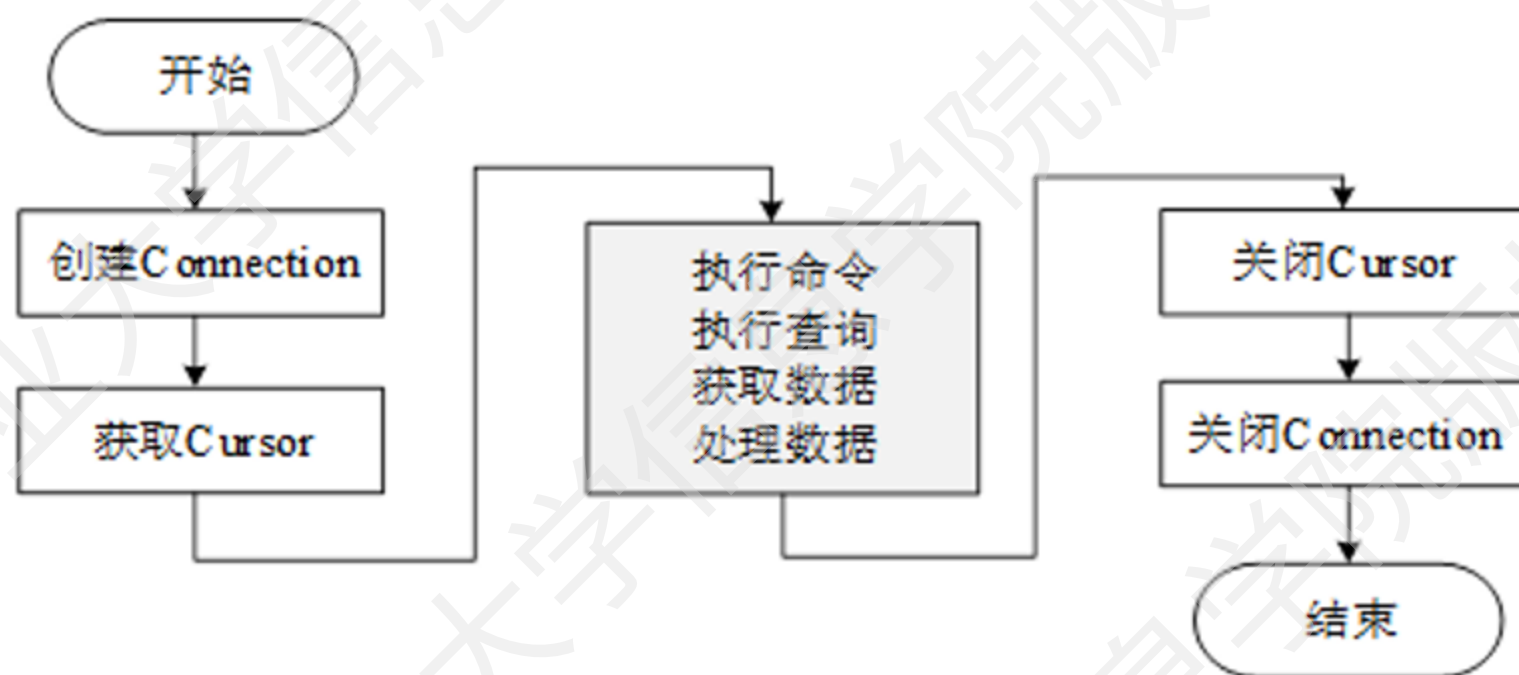
02

PART TWO

API访问数据库

❑ 在Python中，使用Python DB API访问数据库的基本步骤如下所示：

- 导入特定数据库相应的Python编程接口
- 使用connect()函数连接数据库，并返回一个Connection对象
- 通过Connection对象的cursor()方法，返回一个Cursor对象
- 通过Cursor对象的execute()方法执行SQL语句，包括执行命令、查询、获取数据和处理数据等
- 使用connect()函数连接数据库，并返回一个Connection对象
- 通过Connection对象的cursor()方法，返回一个Cursor对象
- 通过Cursor对象的execute()方法执行SQL语句，包括执行命令、查询、获取数据和处理数据等



Python 使用Python DB API访问数据库流程

03

PART Three

连接数据库

连接数据库过程介绍

01

OPTION

导入mysql.connector接口

02

OPTION

连接数据库

03

OPTION

使用游标操作数据库

1.在Python创建的.py文件中导入mysql.connector接口

```
import mysql.connector
```

2.建立与MySQL服务器的连接，并连接当前的数据库

参数说明：

```
db= mysql.connector.connect(  
    host='host_name',  
    user='user_name',  
    passwd='mysql_password',  
    database='database_name',  
    port='port_number')
```

db: 连接对象的返回名。

host_name: MySQL数据库服务器所在的主机名，可以是域名或IP地址

user_name: 可以登录MySQL服务器的用户名。

mysql_password: 登录MySQL数据库服务器验证用户身份密码。

database_name: 要操作的已经创建好的数据库名。

port: 要使用的MySQL端口号，默认通常是3306，一般不需要设置

3.使用游标对连接的数据库对象执行SQL操作和关闭数据库连接

```
cursor = db.cursor()          # 使用数据库连接对象的cursor() 方法创建一个游标对象cursor
try:
    cursor.execute("sql_querys") # 使用 execute() 方法执行 SQL 语句
    db.commit()                 # 提交事务到数据库执行
except:
    db.rollback()               # 如果发生错误则执行回滚操作
    cursor.close()              # 关闭游标
    db.close()                  # 关闭数据库连接
```

说明:

sql_querys: 在MySQL中执行的SQL语句、存储过程或者BEGIN...END语句块

例题：在Python 中连接MySQL数据库的方法connect_database_01.py

```
import mysql.connector
db = mysql.connector.connect(
    host='localhost',
    user='root',
    passwd='123456',
    database='teaching')
cursor = db.cursor()
cursor.execute("SELECT VERSION()")
data = cursor.fetchone()
print("Database version : %s " % data)
cursor.close()
db.close()
```

导入mysql.connector接口
打开数据库连接

使用 cursor() 方法创建一个游标对象 cursor
使用 execute() 方法执行 SQL 查询
使用 fetchone() 方法获取单条数据
输出MySQL的版本
关闭数据库游标
关闭数据库连接

文件connect_database_01.py执行结果显示为 Database version : 8.0.23 ， 即MySQL的版本为8.0.23。



04

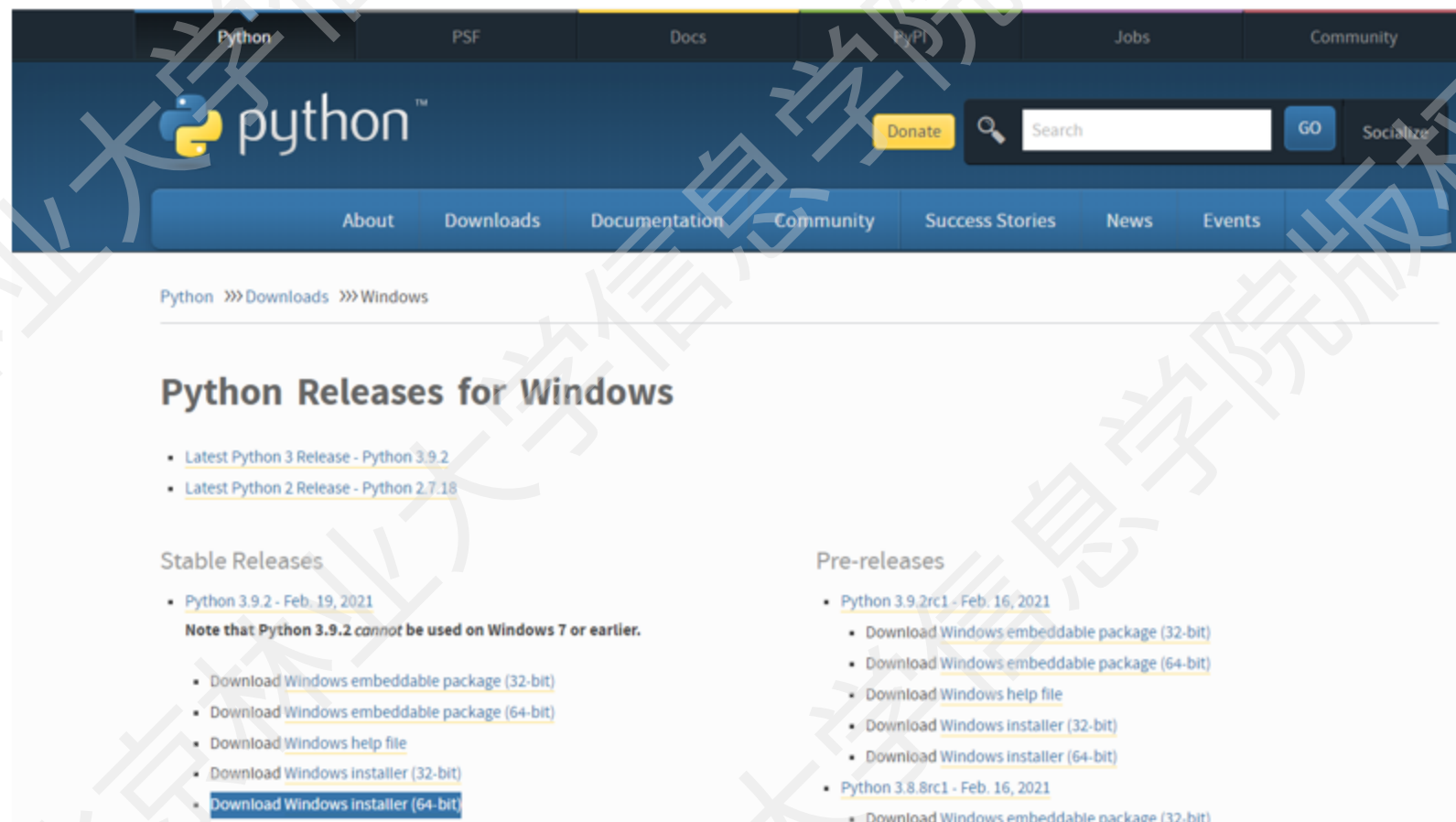
PART Four

Python连接 数据库环境搭建

1. Python的安装

Python的安装比较简单，打开Python官方主页，进入下载界面，选择适合的版本下载并安装即可。如果使用的是Linux系统，则很可能已经预装了某个版本的Python，根据需要进行升级。如果没有特别说明，本章将在Windows 10环境下使用Python 3.9.2进行演示。

步骤1：进入<https://www.python.org/downloads/windows/>网页，选择Download [Windows installer \(64-bit\)](#)直接进行下载。



步骤2：勾选Add Python 3.9 to PATH，后选择Customize installation（自定义安装）进行安装，如图17-4所示。



图17-4 Python 3.9.2安装界面

步骤3：点击Next按钮，之后设置Python安装的路径为D:\Python，如图17-5所示。

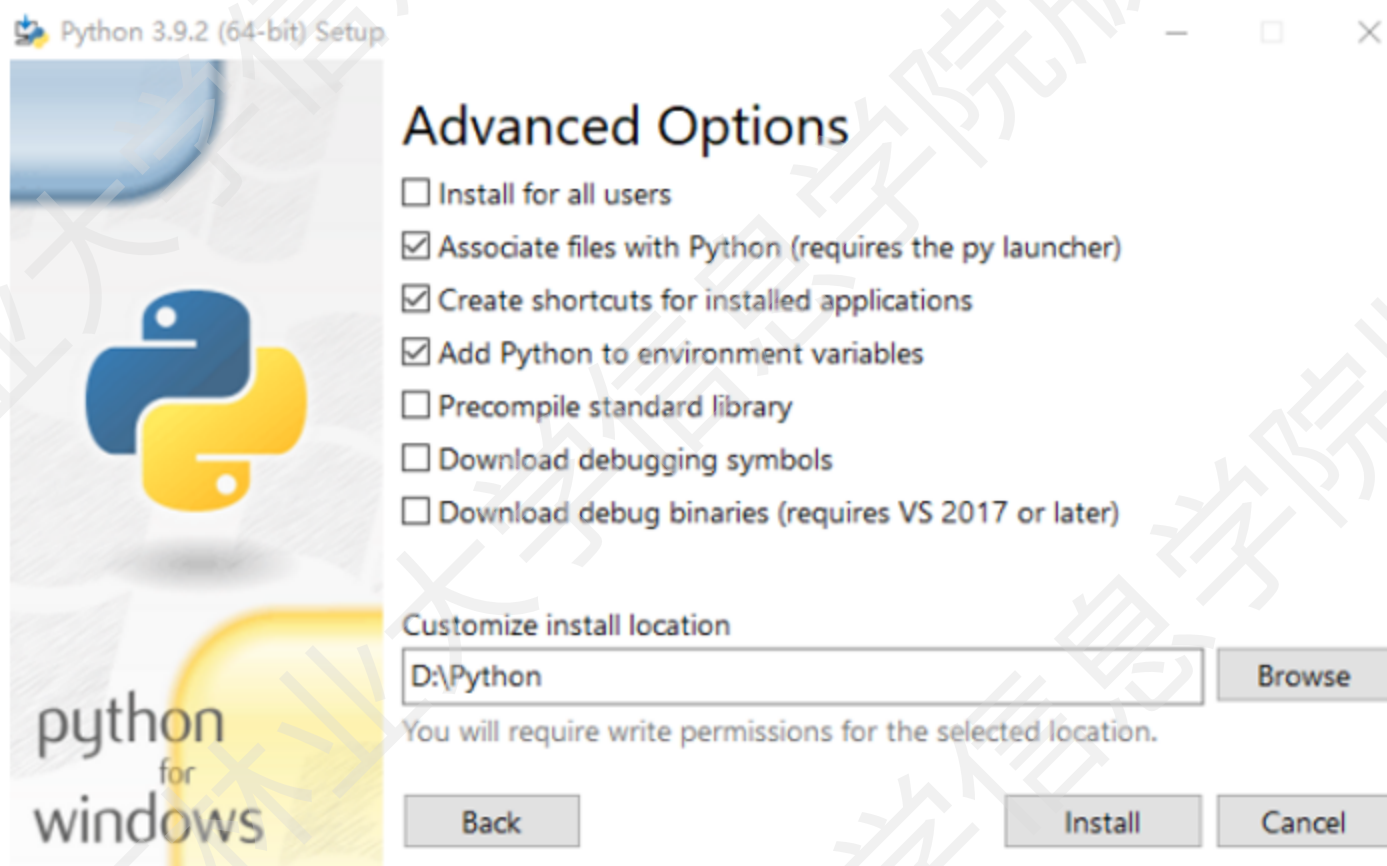
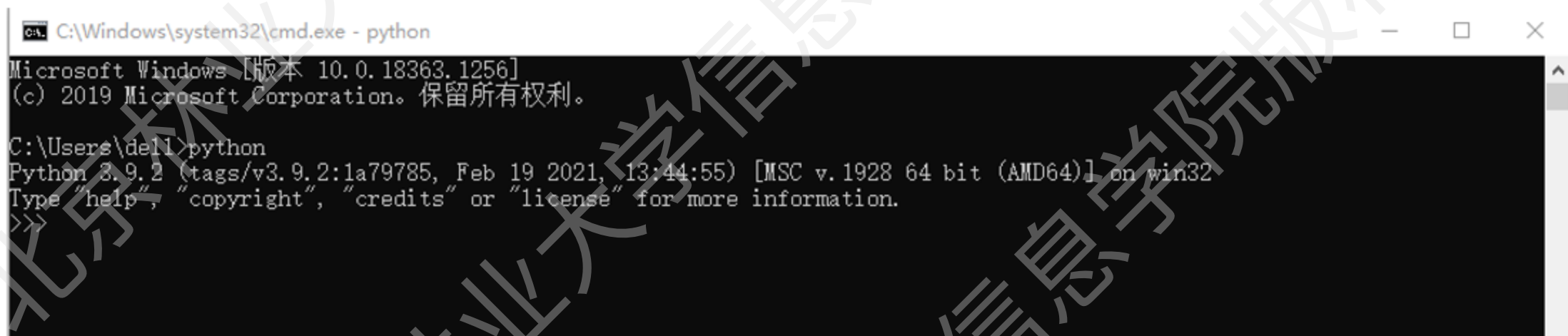


图17-5 自定义Python 3.9.2安装路径

步骤4：安装完毕之后，可以在DOS环境下测试是否安装成功。按Win+R快捷键，输入cmd调出命令提示符，输入python，确认Python安装成功，如图17-6所示，当显示Python版本等信息时，表示Python已经安装成功。



```
C:\Windows\system32\cmd.exe - python
Microsoft Windows [版本 10.0.18363.1256]
(c) 2019 Microsoft Corporation。保留所有权利。

C:\Users\dell>python
Python 3.9.2 (tags/v3.9.2:1a79785, Feb 19 2021, 13:44:55) [MSC v.1928 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

图17-6 Python 安装是否成功测试

2. PyCharm的安装

- ❑ PyCharm 是Python编程语言的集成开发环境。PyCharm和Python的关系相当于Eclipse与Java的关系，即编辑工具与编程语言的关系。PyCharm 包含两个版本，一个是免费的社区版本，另一个是面向企业开发者的高级专业版本。
- ❑ 社区版本：能够实现大部分的功能，包括智能代码补全、直观的项目导航、错误检查和修复、遵循PEP8规范的代码质量检查、智能重构，图形化的调试器和运行器。它还能与IPython notebook进行集成，并支持Anaconda及其他的科学计算包，比如matplotlib和NumPy。
- ❑ 专业版本：除了支持社区版本的功能外，还支持更多高级的功能，比如远程开发功能、数据库支持以及对web开发框架的支持等。由于社区版本不能直接支持MySQL数据库连接，本章采用Python专业版本进行演示。

步骤1：进入链接<https://www.jetbrains.com/pycharm/download/#section=windows>下载Windows版本下的PyCharm专业版本，点击Next按钮进入安装路径设置界面。

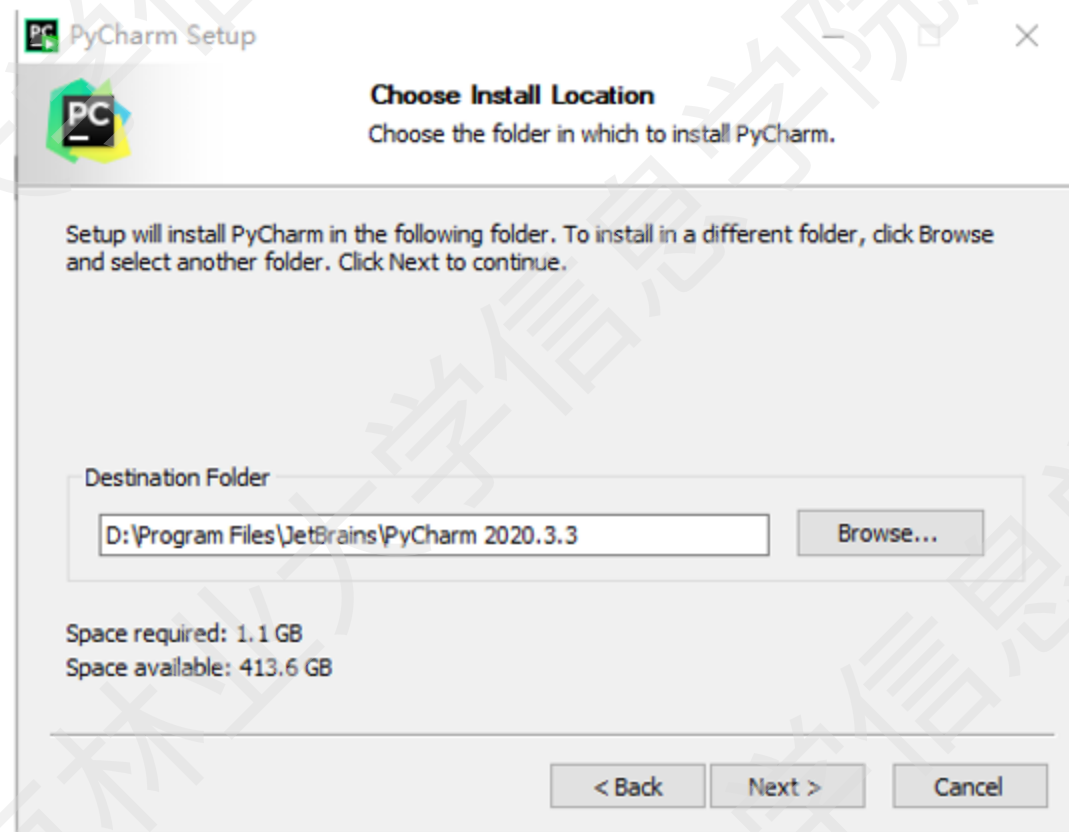


图17-7 PyCharm安装路径设置

步骤2: 进入安装设置界面, 勾选64-bit launcher和.py, 并点击Next按钮进入安装界面。

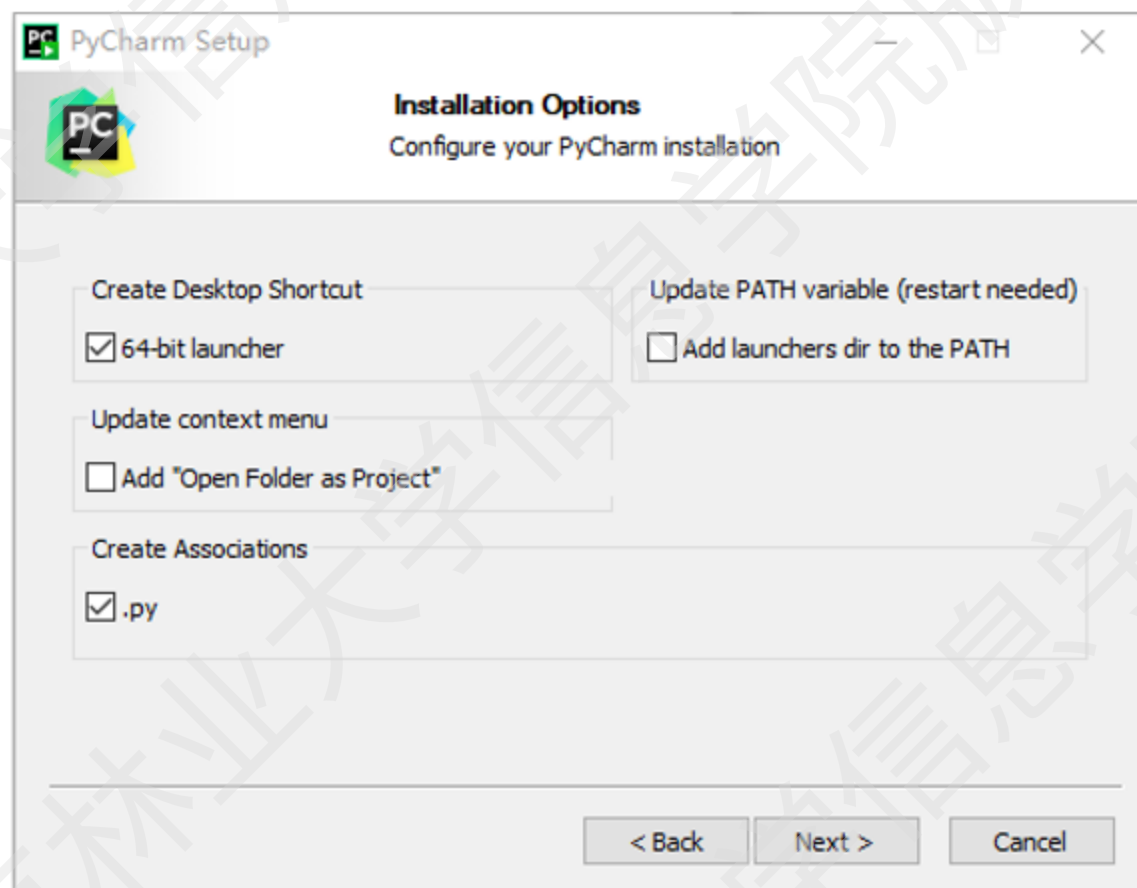


图17-8 PyCharm安装设置

步骤3：点击Install按钮安装PyCharm。

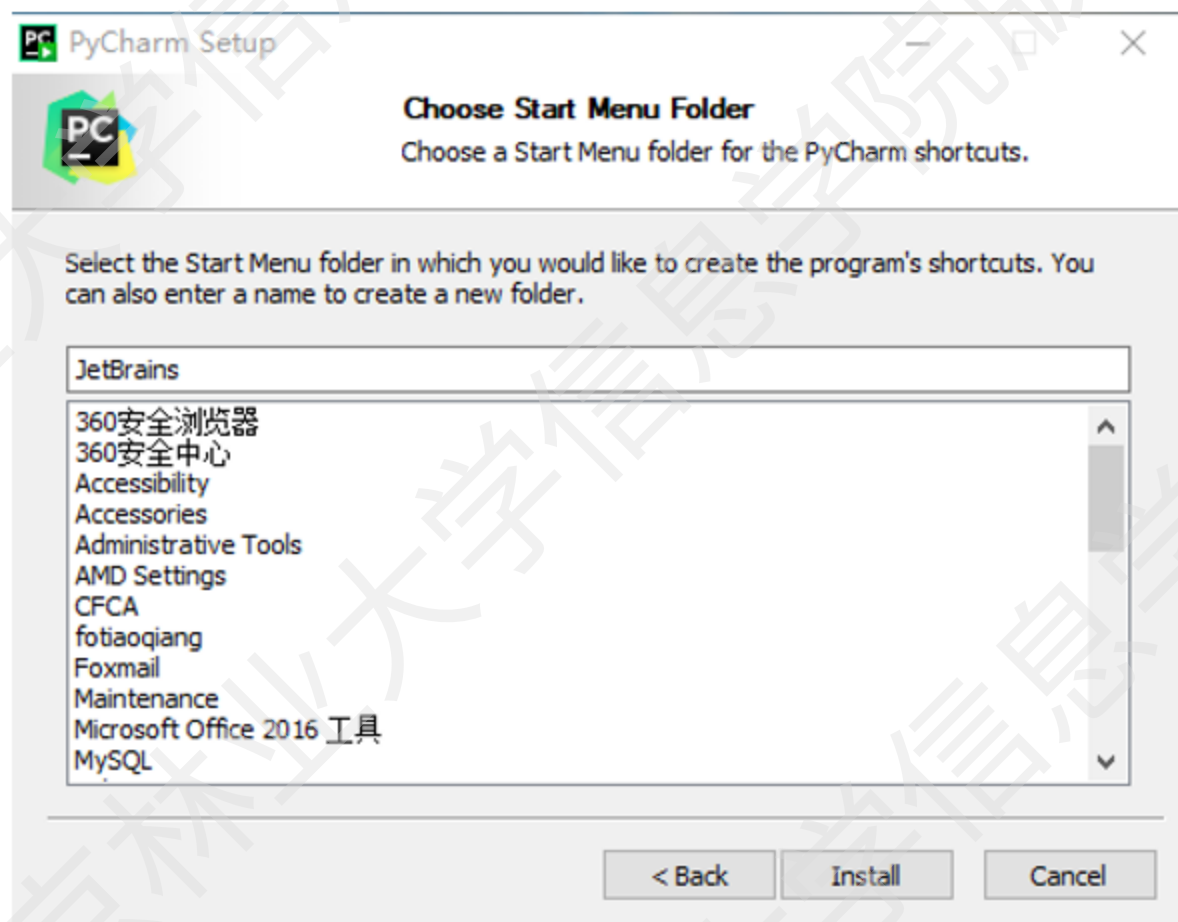


图17-9 PyCharm安装界面

Python连接

MySQL编程实践

本节通过mysql.connector接口演示Python进行MySQL数据库的基本操作，包括数据库的连接，表中数据的插入、修改、删除等操作。本章后续所有的例子都使用第三章中teaching数据库中的基本表。

首先进入网址<https://dev.mysql.com/downloads/connector/python/> 下载mysql-connector-python驱动器。然后在PyCharm中加上mysql-connector-python驱动器。

步骤：

- (1) 点击File->Settings，找到Project:PythonFile目录下的Python Interpreter。
- (2) 点击左下角的+号，搜索找到mysql-connector-python。
- (3) 点击左下角的Install Package按钮将其添加到PyCharm。

CONTENTS

This Is A
List

01

连接数据库

02

操作数据

03

查询数据

04

执行事务

01

PART ONE

连接数据库

连接MySQL中的数据库teaching，并输出MySQL数据库的使用版本，实现实例代码如下（文件名为connect_database_01.py）。

```
import mysql.connector                                #导入mysql.connector接口
db = mysql.connector.connect(                          # 打开数据库连接
    host='localhost',
    user='root',
    passwd='123456',
    database='teaching')
cursor = db.cursor()                                  # 使用 cursor() 方法创建一个游标对象 cursor
cursor.execute("SELECT VERSION()")                   # 使用 execute() 方法执行 SQL 查询
data = cursor.fetchone()                             # 使用 fetchone() 方法获取单条数据.
print("Database version : %s " % data)               # 输出MySQL的版本
cursor.close()                                        # 关闭数游标
db.close()                                            # 关闭数据库连接
```

执行结果显示为 Database version : 8.0.23 ，即MySQL的版本为8.0.23。



02

PART TWO

操作数据

操纵数据基本操作

01

OPTION

插入数据

INSERT

02

OPTION

删除数据

DELETE

03

OPTION

修改数据

UPDATE

示例1：将王天一，苏红霞，林勇和李玉四位同学的信息插入到学生表s中。四位同学的信息如下。

sno	sn	sex	age	maj	dept
s9	王天一	女	18	计算机	信息学院
s10	苏红霞	女	20	信息	信息学院
s11	林勇	男	19	信息	信息学院
s12	李玉	女	21	自动化	工学院

将王天一，苏红霞，林勇和李玉四位同学的信息插入到学生表s中

```
Import mysql.connector  
db = mysql.connector.connect(  
host='localhost',  
user='root',  
passwd='123456',  
database='teaching')
```

打开数据库连接

```
cursor = db.cursor()
```

使用 cursor() 方法创建一个游标对象 cursor

```
try:
```

```
    cursor.execute("INSERT INTO s VALUES('s9','王天一','女',18,'计算机','信息学院')")
```

```
    cursor.execute("INSERT INTO s VALUES('s10','苏红霞','女',20,'信息','信息学院')")
```

```
    cursor.execute("INSERT INTO s VALUES('s11','林勇','男',19,'信息','信息学院')")
```

```
    cursor.execute("INSERT INTO s VALUES('s12','李玉','女',21,'物联网','信息学院')")
```

```
    db.commit()
```

执行sql插入语句

```
except:
```

```
    db.rollback()
```

```
    db.close()
```

提交到数据库执行

如果发生错误则回滚

```
cursor.execute("INSERT INTO s VALUES('s9','王天一','女',18,'计算机','信息学院')")
cursor.execute("INSERT INTO s VALUES('s10','苏红霞','女',20,'信息','信息学院')")
cursor.execute("INSERT INTO s VALUES('s11','林勇','男',19,'信息','信息学院')")
cursor.execute("INSERT INTO s VALUES('s12','李玉','女',21,'物联网','信息学院')")
```

提取 Sql 语句

```
sql1= """"INSERT INTO S VALUES('s9','王天一','女',18,'计算机','信息学院')""""
sql2 = """"INSERT INTO S VALUES('s10','苏红霞','女',20,'信息','信息学院')""""
sql3 = """"INSERT INTO S VALUES('s11','林勇','男',19,'信息','信息学院')""""
sql4 = """"INSERT INTO S VALUES('s12','李玉','女',22,'物联网','信息学院')""""
try:
    cursor.execute(sql1);
    cursor.execute(sql2);
    cursor.execute(sql3);
    cursor.execute(sql4);
    db.commit()
except:
    db.rollback() db.close()
```

等价于执行



通过SELECT关键字查看代码执行的结果，结果如下

sno	sn	sex	age	maj	dept
s1	王彤	女	18	计算机	信息学院
s10	苏红霞	女	20	信息	信息学院
s11	林勇	男	19	信息	信息学院
s12	李玉	女	21	自动化	工学院
s2	苏乐	女	20	信息	信息学院
s3	林毅	男	19	信息	信息学院
s4	陶然	女	18	自动化	工学院
s5	魏立	男	17	数学	理学院
s6	何欣荣	女	21	计算机	信息学院
s7	赵琳琳	女	19	数学	理学院
s8	李轩	男	19	自动化	工学院
s9	王天一	女	18	计算机	信息学院

注意：如果insert_s_02.py文件中插入的记录中存在一条记录的主键在表中已经存在，则文件insert_s_02.py可以再次执行成功，但是不能将文件中需要插入的记录插入到表中。

示例2：将s表中姓名为林毅的maj改为计算机，示例代码如下所示(delete_s_05.py)

```
import mysql.connector
db = mysql.connector.connect(
    host='localhost',
    user='root',
    passwd='123456',
    database='teaching')
```

打开数据库连接

```
cursor = db.cursor()
```

使用 cursor() 方法创建一个游标对象 cursor

```
try:
```

```
    cursor.execute("DELETE FROM s WHERE sno='s12'")
```

执行sql语句

```
    db.commit()
```

```
except:
```

```
    db.rollback()
```

如果发生错误则回滚

```
    db.close()
```

关闭数据库连接

使用SELECT语句查询结果如下

sno	sn	sex	age	maj	dept
s1	王彤	女	18	计算机	信息学院
s10	苏红霞	女	20	信息	信息学院
s11	林勇	男	19	信息	信息学院
s2	苏乐	女	20	信息	信息学院
s3	林毅	男	19	信息	信息学院
s4	陶然	女	18	自动化	工学院
s5	魏立	男	17	数学	理学院
s6	何欣荣	女	21	计算机	信息学院
s7	赵琳琳	女	19	数学	理学院
s8	李轩	男	19	自动化	工学院
s9	王天一	女	18	计算机	信息学院

执行完文件delete_s_04.py的代码之后，我们可以发现s表中李玉的信息已经不见了。

示例3：将s表中姓名为林毅的maj改为计算机，示例代码如下所示(delete_s_05.py)

```
import mysql.connector
db = mysql.connector.connect(
    host='localhost',
    user='root',
    passwd='123456',
    database='teaching')
cursor = db.cursor()
sql="UPDATE s SET maj='计算机' WHERE sn='林毅'"
try:
    cursor.execute(sql)
    db.commit()
except:
    db.rollback()
    db.close()
```

打开数据库连接

使用 cursor() 方法创建一个游标对象 cursor

执行sql语句

如果发生错误则回滚

关闭数据库连接

使用查询语句select * from s where sn='林毅';结果如下：

sno	sn	sex	age	maj	dept
s3	林毅	男	19	计算机	信息学院

03

PART Three

查询数据

示例4：查询s表中age字段大于等于20的所有数据，Python代码如下所示

```
import mysql.connector
db = mysql.connector.connect(
    host='localhost',
    user='root',
    passwd='123456',
    database='teaching')
```

打开数据库连接

```
cursor = db.cursor()
```

使用 cursor() 方法创建一个游标对象 cursor

```
sql = "SELECT * FROM s WHERE AGE >=%s" % 20
```

```
try:
```

执行sql语句

```
    cursor.execute(sql)
```

```
    results = cursor.fetchall()
```

获取所有满足条件的记录列表

```
    print(results)
```

```
except:
```

```
    db.close()
```

关闭数据库连接

代码执行的结果如下：

```
[('s10', '苏红霞', '女', 20, '信息', '信息学院'),
 ('s2', '苏乐', '女', 20, '信息', '信息学院' ),
 ('s6', '何欣荣', '女', 21, '计算机', '信息学院')]
```

示例5：查询s表中age字段小于20的第一条数据，Python代码如下所示

```
import mysql.connector
db = mysql.connector.connect(
    host='localhost',
    user='root',
    passwd='123456',
    database='teaching')
```

打开数据库连接

```
cursor = db.cursor()
```

使用 cursor() 方法创建一个游标对象 cursor

```
sql = "SELECT * FROM s WHERE age < %s" % 20
```

执行sql语句

```
try:
```

```
    cursor.execute(sql)
```

获取所有满足条件的第一条记录

```
    results = cursor.fetchone()
```

```
    print(results)
```

```
except:
```

```
    db.close()
```

关闭数据库连接

执行的结果为：

('s1', '王彤', '女', 18, '计算机', '信息学院')



04

PART Four

执行事务

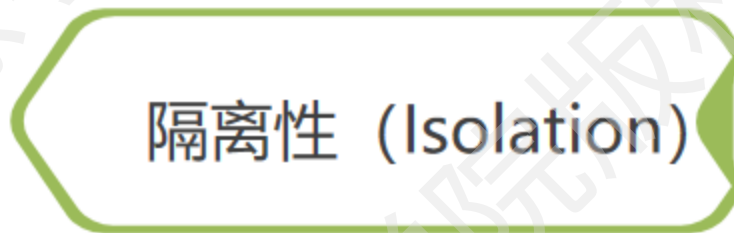
事务 4 个属性



原子性 (Atomicity)



一致性 (Consistency)



隔离性 (Isolation)



持久性 (Durability)

处理事务方法： `commit()`方法：提交游标的所有操作事务
`rollback()`方法：回滚当前游标的所有操作。

- ❑ 在`commit()`方法中，当游标的操作导致数据库改变时遇到`rollback()`方法会回滚到当前游标的所有操作。在对数据库表进行的插入、删除、更新操作时都会对数据进行改变，为确保数据的一致性，当这些操作发生错误时，需要做回滚操作。查询操作不需要修改数据库的数据，因此，不需要执行数据的操作。如果Python程序中不提交`commit()`事务，则游标中的SQL语句操作只在缓冲区得到更改，而在原始表中并未发生改变。例如，在`delete_s_04.py`文件中删除`db.commit()`之后，再执行文件不会改变s表中的数据。

- ❑ Python程序语言的发展过程、使用场景和优势。
- ❑ Python连接数据库的接口Python DB API，使用Python DB API访问数据库的流程以及Python连接MySQL数据库的方法及详细步骤。
- ❑ Python环境的搭建过程。
- ❑ 通过具体的编程实例详细介绍了Python通过mysql.connector接口连接MySQL数据库方法以及通过Python对MySQL数据库数据进行插入、删除、修改和查询等操作的实现方法。
- ❑ Python数据库编程时的两个处理事务的使用方法。



人邮教育

www.ryjiaoyu.com

更多样书申请和资源下载需求，请登录人邮教育社区
(www.ryjiaoyu.com)

海量图书方便查询

囊括各大品类，您想要的应有尽有

免费申请样书

教师免费申请样书，
我们将安排快递迅速送达

下载配套资源

教学视频、PPT课件、教学案例、
习题答案、模拟试卷等丰富资源
免费下载



优惠购书

教师可以申请最低折扣
学生直接优惠购买图书

成为作者

欢迎写文章 / 投稿，我们强大的编辑团队将为您提供专业和高效的编辑出版服务